

CEG 7360 – Embedded Systems

Homework Assignment 1: Timing Diagrams, Testing, and RIMS

Topics: Timing Diagrams, Events, Boolean Logic, Test Vectors, Assertions, and Debugging

Estimated Time: 1.5–2 hours — **Due:** Insert 09/04/2026
— **Submission:** Pilot

Learning Objectives

After completing this assignment, you should be able to:

- Interpret and construct digital timing diagrams.
- Identify rising and falling events and signal pulses.
- Translate Boolean expressions into expected output waveforms.
- Develop systematic test cases for embedded programs.
- Use RIMS to test and debug embedded C programs.
- Create test vectors and assertions to verify program behavior.

Part 1 – Timing Diagram Analysis

Consider the following Boolean expression:

$$B0 = A0 \wedge A1$$

The inputs change according to the following sequence. Each column represents one time interval.

Time Interval	0	1	2	3	4	5	6	7
A0	0	0	1	1	0	1	0	1
A1	0	1	0	1	1	1	0	0

Questions

1.1 Complete the following table by determining the value of $B0$.

Time	A0	A1	B0
0	0	0	
1	0	1	
2	1	0	
3	1	1	
4	0	1	
5	1	1	
6	0	0	
7	1	0	

- 1.2 Draw a timing diagram containing the three signals: $A0$, $A1$, and $B0$. Clearly label the time intervals.
- 1.3 Identify every rising event and falling event on $B0$.
- 1.4 How many pulses does $B0$ contain? Explain your answer briefly.

Part 2 – Boolean Logic and Timing Diagrams

Consider:

$$B0 = A0 \vee A1$$

Use the following input sequence:

Time Interval	0	1	2	3	4	5	6
A0	0	1	0	1	0	1	0
A1	0	0	1	1	0	0	0

Questions

- 2.1 Complete the $B0$ waveform.
- 2.2 Draw the complete timing diagram.
- 2.3 Identify all rising and falling events on $B0$.
- 2.4 Explain why $B0$ remains high when either $A0$ or $A1$ is high.

Part 3 – Exactly-One Condition

Design a system in which $B0$ becomes 1 only when **exactly one** of the two inputs is 1.

In other words: $B0 = 1$ when:

- $A0 = 0, A1 = 1$, or
- $A0 = 1, A1 = 0$.

Questions

- 3.1 Write a Boolean expression for $B0$.
- 3.2 Complete the truth table:

A0	A1	B0
0	0	
0	1	
1	0	
1	1	

- 3.3 Create a timing diagram that covers all four possible input combinations.
- 3.4 Identify the time intervals during which $B0 = 1$.

Part 4 – Events and Triggering

Consider the following system:

```
while (1) {  
    if (A0 == 1)  
        B0 = 1;  
    else  
        B0 = 0;  
}
```

Questions

- 4.1 What type of event occurs when $A0$ changes from 0 to 1?
- 4.2 What type of event occurs when $A0$ changes from 1 to 0?
- 4.3 If $A0$ rises from 0 to 1, what should happen to $B0$?
- 4.4 If $A0$ falls from 1 to 0, what should happen to $B0$?
- 4.5 Explain the difference between a signal event and a pulse.

Part 5 – Testing and Test Vectors

Consider the following RIMS program:

```
#include "RIMS.h"  
  
int main() {  
    while (1) {  
        B0 = A0 && !A1;  
    }  
  
    return 0;  
}
```

Questions

- 5.1 Construct a truth table containing all possible input combinations of $A1A0$.

A1	A0	Expected B0
0	0	
0	1	
1	0	
1	1	

- 5.2 Explain why testing only one or two input combinations is insufficient.
- 5.3 Identify the border cases for this two-input system.
- 5.4 Write RIMS test vectors that test all four possible input combinations. Use the following format:

```
b000000000
wait 100 ms
assert b000000000
```

Your test vector should:

- Set the appropriate inputs.
- Wait at least 100 ms.
- Check the expected output using `assert`.
- Test all four combinations.

Part 6 – Debugging

Writes the following program:

```
#include "RIMS.h"

int main() {
    while (1) {
        B0 = A0 || A1;
    }

    return 0;
}
```

However, if the intended behavior is:

$$B0 = A0 \wedge A1$$

Questions

- 6.1 Identify the programming error.
- 6.2 Provide the corrected line of code.
- 6.3 Give one input combination that would expose this error.
- 6.4 Explain why that input combination exposes the error.
- 6.5 What is the purpose of an assertion when testing embedded software?

Part 7 – RIMS Implementation

Implement the following embedded system in RIMS:

$$B0 = A0 \wedge A1$$

Your program must:

- Continuously evaluate the inputs.
- Set $B0$ according to the Boolean expression.
- Use `printf()` to display the value of $B0$.
- Create a test vector covering all four combinations of $A1A0$.
- Use assertions to verify the expected output.

Your code should include `#include "RIMS.h"` and a continuous `while (1)` loop.

Submit

1. Your RIMS C source code.
2. Your test vector.
3. A screenshot showing the program running successfully.
4. A screenshot of your timing diagram.
5. Your answers to Parts 1–6.

Part 8 – Short Reflection

In 3–5 sentences, answer:

Why is systematic testing important in embedded systems, and why is testing all possible inputs not always practical for larger systems?

Submission Checklist

- ☐ Completed all truth tables.
- ☐ Drawn the required timing diagrams.
- ☐ Identified rising/falling events.
- ☐ Identified pulses.
- ☐ Written the Boolean expressions.
- ☐ Created RIMS test vectors.
- ☐ Used assertions.
- ☐ Tested your program in RIMS.
- ☐ Included screenshots.
- ☐ Completed the short reflection.

Point Breakdown

Section	Points
Part 1 – Timing Diagram Analysis	15
Part 2 – Boolean Logic	10
Part 3 – Exactly-One Condition	15
Part 4 – Events and Triggering	10
Part 5 – Testing/Test Vectors	20
Part 6 – Debugging	10
Part 7 – RIMS Implementation	15
Part 8 – Reflection	5
Total	100

Part 9. Please Attempt all the following questions. Submit your answers via Pilot.

1. Suppose theatrically that we have a computer with a byte addressable has 32 KB memory, provide the following basic needs to the CPU and memory design:
 - What is the largest number that can be brought into the CPU at a time? Give Your answer twice, once in hex and once in decimal.
 - What is the computer's total amount of memory? Give your answer twice, once in bytes and once in megabytes.
 - What is the size of the typical size of address and data bus?
2. What is the difference between volatile and nonvolatile memory? Give example of each.
3. What is a pin? What is a port?
4. What does real-time embedded system mean?
5. What are the typical RAM and ROM capacities for ARM Cortex-M microcontrollers, explain How do they (RAM and ROM) vary depending on the specific microcontroller or SoC model?
6. For the following table, each row represents an equal value expressed in binary, hexadecimal, and decimal. Complete the missing values. Assume the decimal values are unsigned. The first row illustrates the process.

binary	hexadecimal	decimal
2_01101001	0x69	105
	0x48	
		49
2_11001110		
	0xF5	

Suppose you have the following code stored in the following memory locations:

```
0x070001C2          #0x02
0x070001C0    ADDS
0x070001BE          #0x03
0x070001BC    ADDS r2, r3, r0
```

Suppose the code stored in the memory, shown in Hex, represents these 4 instructions. Use the logic in slides 27-33 to show the state of the PC register (initially pointing to 0x070001C2), the values in r1, r2, and r3.

Machine codes are stored in memory

